

Programozási Labor Jegyzetek

C++ Alapok és Objektumorientált Programozás

Labor 1 – Nyelvi alapok

1.1 Mi a C++?

A C++ egy általános célú, magas szintű programozási nyelv, amelyet Bjarne Stroustrup fejlesztett ki a C nyelv kibővítésével. Támogatja:

- procedurális programozást
- objektumorientált programozást
- generikus programozást
- alacsony szintű memóriakezelést

A C++ gyors és hatékony, ezért gyakran használják:

- játékfejlesztésben
 - operációs rendszerekben
 - beágyazott rendszerekben
 - grafikai motorokban
 - nagy teljesítményű alkalmazásokban
-

1.2 Egy egyszerű C++ program

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World!" << endl;
    return 0;
}
```

Magyarázat

```
#include <iostream>
```

Beilleszti az input/output könyvtárat.

```
using namespace std;
```

Lehetővé teszi a standard névtér elemeinek használatát.

```
int main()
```

A program belépési pontja.

```
cout
```

Kimenetre íráshoz használjuk.

```
return 0;
```

Sikeres programfutást jelez.

1.3 Változók

Változó deklaráció

```
int kor;  
float atlag;  
char betu;  
bool igaz;
```

Inicializálás

```
int x = 5;  
float y = 3.14;  
char c = 'A';
```

1.4 Alap adattípusok

Típus	Jelentés	Példa
int	egész szám	5
float	lebegőpontos	3.14
double	dupla pontosság	2.71828
char	karakter	'A'
bool	logikai	true

Típus	Jelentés	Példa
string	szöveg	"alma"

1.5 Konstansok

```
const double PI = 3.14159;
```

A konstans értéke nem módosítható.

1.6 Operátorok

Aritmetikai operátorok

Operátor	Jelentés
+	összeadás
-	kivonás
*	szorzás
/	osztás
%	maradék

Példa

```
int a = 10;
int b = 3;

cout << a + b << endl;
cout << a % b << endl;
```

Relációs operátorok

Operátor	Jelentés
==	egyenlő
!=	nem egyenlő
>	nagyobb
<	kisebb

Operátor	Jelentés
>=	nagyobb vagy egyenlő
<=	kisebb vagy egyenlő

Logikai operátorok

Operátor	Jelentés
&&	és
!	negálás

1.7 Input és output

Bemenet

```
int x;  
cin >> x;
```

Kimenet

```
cout << x;
```

Több adat kiírása

```
cout << "A szám: " << x << endl;
```

1.8 Típuskonverzió

Implicit

```
int x = 5;  
double y = x;
```

Explicit

```
double x = 3.9;
int y = (int)x;
```

1.9 Gyakorló példák

Két szám összege

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;
    cin >> a >> b;

    cout << a + b;

    return 0;
}
```

Téglalap kerülete és területe

```
#include <iostream>
using namespace std;

int main()
{
    int a, b;

    cin >> a >> b;

    cout << "Kerület: " << 2 * (a + b) << endl;
    cout << "Terület: " << a * b << endl;

    return 0;
}
```

Labor 2 – Vezérlési szerkezetek

2.1 Elágazások

if

```
if (feltetel)
{
    utasitas;
}
```

Példa

```
int x;
cin >> x;

if (x > 0)
{
    cout << "Pozitiv";
}
```

if-else

```
if (x % 2 == 0)
{
    cout << "Paros";
}
else
{
    cout << "Paratlan";
}
```

Több ág

```
if (jegy == 5)
{
    cout << "Jeles";
}
else if (jegy == 4)
{
    cout << "Jo";
}
```

```
}  
else  
{  
    cout << "Elegtelen";  
}
```

2.2 switch

```
switch(n)  
{  
    case 1:  
        cout << "Hetfo";  
        break;  
  
    case 2:  
        cout << "Kedd";  
        break;  
  
    default:  
        cout << "Ismeretlen";  
}
```

2.3 Ciklusok

while ciklus

```
while (feltetel)  
{  
    utasitasok;  
}
```

Példa

```
int i = 1;  
  
while(i <= 5)  
{  
    cout << i << endl;  
    i++;  
}
```

do-while

```
do
{
    utasitasok;
}
while(feltetel);
```

for ciklus

```
for(inicializalas; feltetel; modositas)
{
    utasitasok;
}
```

Példa

```
for(int i = 1; i <= 10; i++)
{
    cout << i << endl;
}
```

2.4 break és continue

break

Megszakítja a ciklust.

```
for(int i = 0; i < 10; i++)
{
    if(i == 5)
        break;

    cout << i << endl;
}
```

continue

A ciklus következő iterációjára ugrik.

```
for(int i = 0; i < 10; i++)
{
    if(i == 5)
        continue;

    cout << i << endl;
}
```

2.5 Beágyazott ciklusok

```
for(int i = 1; i <= 3; i++)
{
    for(int j = 1; j <= 3; j++)
    {
        cout << i << " " << j << endl;
    }
}
```

2.6 Gyakorló feladatok

Számjegyek összege

```
int n;
cin >> n;

int osszeg = 0;

while(n > 0)
{
    osszeg += n % 10;
    n /= 10;
}

cout << osszeg;
```

Faktoriális

```
int n;  
cin >> n;  
  
int fakt = 1;  
  
for(int i = 1; i <= n; i++)  
{  
    fakt *= i;  
}  
  
cout << fakt;
```

Labor 3 – Pointerek

3.1 Memória alapok

A változók a memóriában tárolódnak.

Minden változónak van:

- értéke
- memóriacíme

3.2 Pointer fogalma

A pointer memóriacímet tárol.

```
int x = 10;  
int* p = &x;
```

3.3 Operátorok

&

Címlekérő operátor.

```
cout << &x;
```

*

Dereferáló operátor.

```
cout << *p;
```

3.4 Pointer példa

```
#include <iostream>
using namespace std;

int main()
{
    int x = 5;
    int* p = &x;

    cout << x << endl;
    cout << &x << endl;
    cout << p << endl;
    cout << *p << endl;

    return 0;
}
```

3.5 Pointer módosítás

```
int x = 10;
int* p = &x;

*p = 20;

cout << x;
```

3.6 Nullpointer

```
int* p = nullptr;
```

3.7 Dinamikus memórafoglalás

new

```
int* p = new int;
```

delete

```
delete p;
```

3.8 Dinamikus tömb

```
int n;  
cin >> n;  
  
int* tomb = new int[n];  
  
for(int i = 0; i < n; i++)  
{  
    cin >> tomb[i];  
}  
  
for(int i = 0; i < n; i++)  
{  
    cout << tomb[i] << " ";  
}  
  
delete[] tomb;
```

3.9 Pointeraritmetika

```
int tomb[3] = {10,20,30};
int* p = tomb;

cout << *p << endl;
cout << *(p+1) << endl;
```

3.10 Pointer és függvény

```
void novel(int* x)
{
    (*x)++;
}

int main()
{
    int a = 5;

    novel(&a);

    cout << a;
}
```

Labor 4 – Függvények és eljárások

4.1 Függvény fogalma

A függvény újrafelhasználható kódrészlet.

4.2 Függvény szintaxis

```
visszateresi_tipus nev(parameterek)
{
    utasitasok;
}
```

4.3 Egyszerű függvény

```
int osszeg(int a, int b)
{
    return a + b;
}
```

4.4 Függvényhívás

```
cout << osszeg(2,3);
```

4.5 Paraméterátadás

Érték szerinti

```
void f(int x)
{
    x++;
}
```

Az eredeti változó nem módosul.

Referencia szerinti

```
void f(int& x)
{
    x++;
}
```

Az eredeti változó módosul.

Pointer szerinti

```
void f(int* x)
{
```

```
    (*x)++;  
}
```

4.6 Függvénytúterhelés

```
int osszeg(int a, int b)  
{  
    return a + b;  
}  
  
float osszeg(float a, float b)  
{  
    return a + b;  
}
```

4.7 Rekurzió

A függvény önmagát hívja.

Faktoriális

```
int fakt(int n)  
{  
    if(n == 0)  
        return 1;  
  
    return n * fakt(n-1);  
}
```

4.8 Scope

Lokális változó

Csak a blokkban látható.

Globális változó

Minden függvényből elérhető.

4.9 Inline függvény

```
inline int negyzet(int x)
{
    return x*x;
}
```

Labor 5 – Struktúrák

5.1 Struktúra fogalma

A struktúra több adatot fog össze.

5.2 Struktúra deklaráció

```
struct Diak
{
    string nev;
    int kor;
    double atlag;
};
```

5.3 Struktúra használata

```
Diak d;

d.nev = "Anna";
d.kor = 20;
d.atlag = 4.5;
```

5.4 Struktúra kiírás

```
cout << d.nev << endl;
cout << d.kor << endl;
```


5.5 Tömb struktúrából

```
Diak tomb[3];
```

5.6 Struktúra függvényparaméterként

```
void kiir(Diak d)
{
    cout << d.nev;
}
```

5.7 Referenciás átadás

```
void novelKor(Diak& d)
{
    d.kor++;
}
```

5.8 Pointer struktúrára

```
Diak d;
Diak* p = &d;

p->kor = 21;
```

5.9 Példa program

```
#include <iostream>
using namespace std;

struct Auto
{
    string marka;
    int ev;
};
```

```
int main()
{
    Auto a;

    a.marka = "BMW";
    a.ev = 2020;

    cout << a.marka << endl;
    cout << a.ev << endl;

    return 0;
}
```

Labor 6 – Gyakorlás, struktúrák

6.1 Diák nyilvántartás

```
struct Diak
{
    string nev;
    int kor;
    double atlag;
};
```

6.2 Több diák kezelése

```
Diak diakok[100];
```

6.3 Beolvasás

```
for(int i = 0; i < n; i++)
{
    cin >> diakok[i].nev;
    cin >> diakok[i].kor;
    cin >> diakok[i].atlag;
}
```

6.4 Keresés

```
for(int i = 0; i < n; i++)
{
    if(diakok[i].nev == "Anna")
    {
        cout << diakok[i].atlag;
    }
}
```

6.5 Rendezés

```
for(int i = 0; i < n-1; i++)
{
    for(int j = i+1; j < n; j++)
    {
        if(diakok[i].atlag > diakok[j].atlag)
        {
            swap(diakok[i], diakok[j]);
        }
    }
}
```

6.6 Maximum keresés

```
int maxi = 0;

for(int i = 1; i < n; i++)
{
    if(diakok[i].atlag > diakok[maxi].atlag)
    {
        maxi = i;
    }
}
```

Labor 7 – Osztályok, konstruktorok, getterek és setterek

7.1 Objektumorientált programozás

Fő fogalmak:

- osztály
 - objektum
 - öröklődés
 - polimorfizmus
 - enkapszuláció
-

7.2 Osztály

```
class Auto
{
private:
    string marka;
    int ev;

public:
    void setEv(int e)
    {
        ev = e;
    }

    int getEv()
    {
        return ev;
    }
};
```

7.3 Objektum létrehozása

```
Auto a;
```

7.4 Konstruktor

A konstruktor automatikusan lefut objektum létrehozásakor.

```
class Auto
{
private:
    string marka;

public:
    Auto()
    {
        marka = "Ismeretlen";
    }
};
```

7.5 Paraméteres konstruktor

```
class Auto
{
private:
    string marka;

public:
    Auto(string m)
    {
        marka = m;
    }
};
```

7.6 Getter és setter

Setter

```
void setMarka(string m)
{
    marka = m;
}
```

Getter

```
string getMarka()
{
    return marka;
}
```

7.7 this pointer

```
class Diak
{
private:
    int kor;

public:
    void setKor(int kor)
    {
        this->kor = kor;
    }
};
```

7.8 Destruktor

```
~Auto()
{
    cout << "Objektum torolve";
}
```

7.9 Komplette példa

```
#include <iostream>
using namespace std;

class Szemely
{
private:
    string nev;
    int kor;
```

```

public:
    Szemely(string n, int k)
    {
        nev = n;
        kor = k;
    }

    void kiir()
    {
        cout << nev << " " << kor << endl;
    }
};

int main()
{
    Szemely s("Peter", 20);
    s.kiir();

    return 0;
}

```

Labor 8 – Öröklődés

8.1 Öröklődés fogalma

Az egyik osztály átveszi egy másik osztály tulajdonságait.

8.2 Alaposztály és származtatott osztály

```

class Allat
{
public:
    void hangotAd()
    {
        cout << "Hang";
    }
};

class Kutya : public Allat
{
};

```

8.3 Példa

```
Kutya k;  
k.hangotAd();
```

8.4 Konstruktor öröklődés

```
class Szemely  
{  
protected:  
    string nev;  
  
public:  
    Szemely(string n)  
    {  
        nev = n;  
    }  
};  
  
class Diak : public Szemely  
{  
private:  
    int jegy;  
  
public:  
    Diak(string n, int j) : Szemely(n)  
    {  
        jegy = j;  
    }  
};
```

8.5 protected

A protected tag:

- a leszármazott osztályból elérhető
- kívülről nem

8.6 Függvény felülírás

```
class Allat
{
public:
    void hang()
    {
        cout << "Allat hang";
    }
};

class Macska : public Allat
{
public:
    void hang()
    {
        cout << "Miau";
    }
};
```

8.7 Virtuális függvény

```
class Allat
{
public:
    virtual void hang()
    {
        cout << "Allat";
    }
};
```

8.8 Polimorfizmus

```
Allat* a;
Macska m;

a = &m;
a->hang();
```

Labor 9 – String osztály

9.1 string használata

```
string s = "alma";
```

9.2 Beolvasás

Egy szó

```
cin >> s;
```

Egész sor

```
getline(cin, s);
```

9.3 String műveletek

Hossz

```
s.length()
```

Összefűzés

```
string a = "piros";  
string b = "alma";  
  
cout << a + b;
```

Karakter elérés

```
cout << s[0];
```

Részstring

```
s.substr(1,3)
```

Keresés

```
s.find("abc")
```

9.4 String bejárás

```
for(int i = 0; i < s.length(); i++)  
{  
    cout << s[i] << endl;  
}
```

9.5 Karaktervizsgálat

```
isdigit(c)  
isalpha(c)  
toupper(c)  
tolower(c)
```

9.6 Példa – magánhangzók száma

```
string s;  
getline(cin, s);  
  
int db = 0;  
  
for(int i = 0; i < s.length(); i++)  
{  
    char c = tolower(s[i]);  
  
    if(c=='a' || c=='e' || c=='i' || c=='o' || c=='u')  
    {
```

```
        db++;  
    }  
}  
  
cout << db;
```

Labor 10 – Fájlkezelés

10.1 Fájlkezelő könyvtár

```
#include <fstream>
```

10.2 Fájlba írás

```
ofstream fajl("adat.txt");  
  
fajl << "Hello";  
  
fajl.close();
```

10.3 Fájlból olvasás

```
ifstream fajl("adat.txt");  
  
string s;  
  
fajl >> s;  
  
cout << s;  
  
fajl.close();
```

10.4 Soronkénti olvasás

```
string sor;
```

```
while(getline(fajl, sor))
{
    cout << sor << endl;
}
```

10.5 Több adat olvasása

```
int x;

while(fajl >> x)
{
    cout << x << endl;
}
```

10.6 Példa – számok összege fájlból

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream fajl("szamok.txt");

    int x;
    int osszeg = 0;

    while(fajl >> x)
    {
        osszeg += x;
    }

    cout << osszeg;

    fajl.close();

    return 0;
}
```

10.7 append mód

```
ofstream fajl("adat.txt", ios::app);
```

10.8 Bináris fájlok

```
ofstream fajl("adat.bin", ios::binary);
```

Labor 11 – Operátor- és metódustúlterhelés, template-ek

11.1 Operátortúlterhelés

Lehetővé teszi operátorok saját definícióját.

11.2 + operátor túlterhelése

```
class Pont
{
public:
    int x, y;

    Pont(int a, int b)
    {
        x = a;
        y = b;
    }

    Pont operator+(Pont p)
    {
        return Pont(x + p.x, y + p.y);
    }
};
```

11.3 Használat

```
Pont a(1,2);  
Pont b(3,4);  
  
Pont c = a + b;
```

11.4 << operátor túlterhelése

```
class Diak  
{  
public:  
    string nev;  
  
    friend ostream& operator<<(ostream& os, Diak d)  
    {  
        os << d.nev;  
        return os;  
    }  
};
```

11.5 == operátor

```
bool operator==(Pont p)  
{  
    return x == p.x && y == p.y;  
}
```

11.6 Metódustúlterhelés

```
class Kiir  
{  
public:  
    void f(int x)  
    {  
        cout << x;  
    }  
  
    void f(string s)
```

```
    {  
        cout << s;  
    }  
};
```

11.7 Template-ek

A template generikus programozást tesz lehetővé.

11.8 Függvény template

```
template <typename T>  
T maximum(T a, T b)  
{  
    if(a > b)  
        return a;  
  
    return b;  
}
```

11.9 Használat

```
cout << maximum(3,5);  
cout << maximum(2.5,7.1);
```

11.10 Osztály template

```
template <class T>  
class Tarolo  
{  
private:  
    T adat;  
  
public:  
    Tarolo(T a)  
    {  
        adat = a;  
    }  
}
```



```
T getAdat()
{
    return adat;
}
};
```

11.11 Template specializáció

```
template<>
class Tarolo<char>
{
private:
    char adat;

public:
    Tarolo(char a)
    {
        adat = toupper(a);
    }
};
```

Összefoglalás

A kurzus legfontosabb témái

Labor	Téma
1	Nyelvi alapok
2	Vezérlési szerkezetek
3	Pointerek
4	Függvények
5	Struktúrák
6	Struktúra gyakorlás
7	Osztályok
8	Öröklődés
9	String
10	Fájlkezelés

Labor	Téma
11	Operátortúlterhelés, template

Tanulási tippek

1. Minden témából írd saját példaprogramot.
2. Gyakorold a hibakeresést.
3. Használj sok ciklust és függvényt.
4. Pointereknél figyelj a memóriaszivárgásra.
5. OOP témáknál gyakorold az osztálytervezést.
6. Template-eknél próbálj több adattípust.
7. Fájlkezelésnél mindig zárd le a fájlt.

Gyakori hibák

Elfelejtett pontosvessző

```
int x = 5
```

Nullpointer dereferálás

```
int* p = nullptr;  
cout << *p;
```

Végtelen ciklus

```
while(true)  
{  
}
```

Memóriaszivárgás

```
int* p = new int;
```

Nincs delete.

Hasznos standard könyvtárak

Könyvtár	Funkció
iostream	Input/output
string	Sztringek
fstream	Fájlkezelés
cmath	Matematikai függvények
vector	Dinamikus tömb
algorithm	Algoritmusok

Extra gyakorló feladatok

1. Prímszám ellenőrzés
 2. Fibonacci sorozat
 3. Mátrix összeadás
 4. Szöveg visszafelé kiírása
 5. Diáknyilvántartó rendszer
 6. Banki számla osztály
 7. Öröklődéses jármű rendszer
 8. Fájlba mentett jegyzék
 9. Saját vector osztály
 10. Template alapú maximum keresés
-

Vizsgafelkészülési tanácsok

- Tanuld meg az alap szintaxist.
 - Gyakorolj sok kis programot.
 - Értsd a pointereket.
 - Ismerd az OOP alapelveket.
 - Tudd mikor kell referencia vagy pointer.
 - Gyakorold a fájlkezelést.
 - Értsd a template-ek működését.
-

Vége